

Reviewer Pack — Pullback monotonicity of coarse depth under coarse-Lipschitz...

Entry cc8127953d91 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 19:34:39 UTC

Pullback monotonicity of coarse depth under coarse-Lipschitz reductions: a functoriality test for κ in CG-KW

Entry ID: cc8127953d91 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 19:34:39 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW) framework; underlying math branch: large-scale / coarse geometry of finite metric spaces and Roe C^* -algebras (Roe 2003; Nowak-Yu 2012), interfaced with Karchmer-Wigderson communication geometry **Field B** (complexity object): Boolean formula depth (NC^1 vs. P) under m -reductions: specifically, the behavior of formula-depth lower bounds along coarse-Lipschitz many-one reductions $\varphi: \{0,1\}^n \rightarrow \{0,1\}^{n'}$ between explicit functions $f \leq_m g$ (a complexity-theoretic preorder on Boolean functions)

Statement:

Let $\varphi: \{0,1\}^n \rightarrow \{0,1\}^{n'}$ witness a many-one reduction $f \leq_m g$, and suppose φ induces a coarse-Lipschitz map $\Phi: (X_f, d_f) \rightarrow (X_g, d_g)$ with distortion $D := \sup_{x \neq y} \max(d_g(\Phi x, \Phi y)/d_f(x, y), d_f(x, y)/d_g(\Phi x, \Phi y))$. Then the reduction pullback $\Phi^*: C_u[X_g] \rightarrow C_u[X_f]$ and $\Phi^*: HX^1(X_g) \rightarrow HX^1(X_f)$ satisfy the inequality $\kappa(f) \leq D \cdot \kappa(g) + O(1)$. Consequently, the CG-KW target invariant κ is (up-to-distortion) monotone under the m -reduction preorder, so a super-logarithmic κ -lower bound for any g transfers to every f that coarse-Lipschitz reduces to g , yielding an oracle-free, NP-hardness-style closure for κ -based depth lower bounds.

Rationale (proposer's reasoning):

This sub-conjecture stress-tests A5 (Roe-index rigidity) together with the categorical glue provided by the reduction pullback operation. A5 asserts that Roe-trace classes are coarse-equivalence invariants; a reduction with bounded distortion is a quantitative coarse map, so the index pairing (c, T) should pull back with at most a multiplicative D -loss — exactly mirroring how Mendel-Naor metric-cotype invariants and Yu's coarse Baum-Connes invariants behave under quasi-isometries. If the inequality fails experimentally, A5 is false at finite scale; if it holds, κ inherits the m -reduction closure that classical complexity invariants enjoy, which is essential for using CG-KW to lift bounds for one explicit function to many.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 51abd077f3b327e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across the 4 prescribed (f,g) pairs, run with 5 random Roe-operator seeds each (20 trials).
For every trial compute $\text{slack} = D \cdot \kappa(g) + 1 - \kappa(f)$. PASS iff $\text{slack} \geq 0$ in $\geq 17/20$ trials AND
mean $\text{slack} \geq 0$ AND ≥ 3 pairs have non-vacuous $\kappa(g) \geq 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.78	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.82	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse geometry Karchmer-Wigderson formula depth lower bounds - Roe algebra uniform Roe C* Boolean function complexity reduction - coarse Lipschitz many-one reduction monotone NC1 depth invariant

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/2002.07444v1] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [http://arxiv.org/abs/1404.7443v2] Depth Lower Bounds against Circuits with Sparse Orientation - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/2507.06261v6] Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capab - [http://arxiv.org/abs/2109.09255v2] Search for continuous gravitational waves from 20 accreting millisecond X-ray pulsars in O3 LIGO data - [http://arxiv.org/abs/cond-mat/0602024v2] Basic Types of Coarse-Graining - [http://arxiv.org/abs/1803.07099v3] Varieties of coarse spaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

# Define candidate pairs (f, g)
candidates = [
    (([0]*3 + [1]*3), ([0]*5 + [1]*5)),
    (([0]*4 + [1]*2), ([0]*5 + [1]*5)),
    (([0]*4 + [1]*2), ([0]*6 + [1]*6)),
    (([0]*4 + [1]*2), ([0]*5 + [1]*5))
]

# Function to compute the KW-metric
def kw_metric(f, g):
    n = len(f)
    n_prime = len(g)
    metric = 0
    for x in range(2**n):
        for y in range(2**n_prime):
            if f[x] != f[y]:
                d_f = abs(x - y)
                d_g = abs(g[f.index(x)] - g[f.index(y)])
                metric = max(metric, max(d_g / d_f, d_f / d_g))
    return metric

# Function to compute the Roe index
def roe_index(f, R):
    n = len(f)
    metric = kw_metric(f, f)
    count = 0
    for x in range(2**n):
        if f[x].count(1) == 1:
            count += 1
    return math.log(count) / math.log(R)

# Function to compute the pullback Roe index
def pullback_roe_index(f, g, phi, R):
    n = len(f)
    n_prime = len(g)
    metric_g = kw_metric(g, g)
    count_g = 0
    for x in range(2**n_prime):
        if g[x].count(1) == 1:
            count_g += 1

    count_f = 0
    for x in range(2**n):
        if f[phi(x)].count(1) == 1:
            count_f += 1

    return math.log(count_f) / math.log(R * metric_g)

# Function to compute the empirical distortion
def empirical_distortion(f, g, phi):
    n = len(f)
    n_prime = len(g)
    max_dist = 0
    for x in range(2**n):
        for y in range(2**n_prime):
            if f[x] != f[y]:

```

```

        d_f = abs(x - y)
        d_g = abs(g[phi[f.index(x)]] - g[phi[f.index(y)]])
        max_dist = max(max_dist, max(d_g / d_f, d_f / d_g))
    return max_dist

# Function to compute the pullback Roe operator
def pullback_roe_operator(f, g, phi, T):
    n = len(f)
    n_prime = len(g)
    R = len(T)
    pulled_back_T = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if f[i].count(1) == 1 and f[j].count(1) == 1:
                pulled_back_T[phi[i]][phi[j]] = T[i][j]
    return pulled_back_T

# Function to compute the pullback Roe index with a Roe operator
def pullback_roe_index_with_operator(f, g, phi, R, T):
    n = len(f)
    n_prime = len(g)
    metric_g = kw_metric(g, g)
    count_g = 0
    for x in range(2**n_prime):
        if g[x].count(1) == 1:
            count_g += 1

    pulled_back_T = pullback_roe_operator(f, g, phi, T)

    count_f = 0
    for i in range(n):
        for j in range(n):
            if f[i].count(1) == 1 and f[j].count(1) == 1:
                count_f += pulled_back_T[i][j]

    return math.log(count_f) / math.log(R * metric_g)

# Function to compute the empirical slack
def empirical_slack(f, g, phi, R):
    n = len(f)
    n_prime = len(g)
    max_dist = empirical_distortion(f, g, phi)
    kappa_g = roe_index(g, R)
    kappa_f = pullback_roe_index_with_operator(f, g, phi, R, [[1]*n for _ in range(n)])
    return max_dist * kappa_g + 1 - kappa_f

# Main trial logic
results = []
for f, g in candidates:
    n = len(f)
    n_prime = len(g)

    # Compute the empirical distortion
    phi = [i % n_prime for i in range(n)]
    max_dist = empirical_distortion(f, g, phi)

```

```

# Compute the Roe index and pullback Roe index
R = 2
kappa_g = roe_index(g, R)
kappa_f = pullback_roe_index_with_operator(f, g, phi, R, [[1]*n for _ in range(n)])

# Compute the empirical slack
slack = max_dist * kappa_g + 1 - kappa_f

results.append({
    "f": f,
    "g": g,
    "phi": phi,
    "max_dist": max_dist,
    "kappa_g": kappa_g,
    "kappa_f": kappa_f,
    "slack": slack
})

# Compute the mean and standard deviation of the slack
mean_slack = sum(result["slack"] for result in results) / len(results)
std_slack = math.sqrt(sum((result["slack"] - mean_slack)**2 for result in results) / len(results))

# Check if the conjecture holds
conjecture_holds = all(result["slack"] >= 0 for result in results)

return {
    "metric_name": "empirical_slack",
    "metric_value": mean_slack,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else str(results[0])
}

if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if len(sys.argv) == 1 else [int(seed) for seed in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": {"seed": seed, **result}}))
        results.append(result)

    mean_slack = sum(result["metric_value"] for result in results) / len(results)
    std_slack = math.sqrt(sum((result["metric_value"] - mean_slack)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std={std_slack} support_fraction={support_fraction}")
    elif sum(1 for result in results if result["slack"] >= 0) / len(results) >= 0.85:
        print(f"RESULT: SUPPORTED mean={mean_slack} std={std_slack} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c3d4406.py", line 187, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5c3d4406.py", line 100, in run_trial
    if f[x].count(1) == 1 and f[y].count(1) == 1:
                                   ~~~
IndexError: tuple index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any trial data, so the pre-registered support condition could not be evaluated. Per the rules, a crashed test yields INCONCLUSIVE. | next: Fix the indexing bug in run_trial (line 100 accesses f[y] out of range — likely a mismatch between tuple length and sampled indices) and rerun the 20-trial protocol.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	6301
2	novelty	claude_max	opus	0	0	3592
3	novelty	claude_max	opus	0	0	9801
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32152
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19723
6	judge	claude_max	opus	0	0	4628

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 76197 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/cc8127953d91.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cc8127953d91.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cc8127953d91.tar.gz` (if generated)